Determining what goes on-chain and what stays off-chain is a significant design decision that makes blockchain application development different from traditional application development. In a traditional application, there is no on-chain data because *there is no blockchain*. The blockchain is a whole new concrete addition to enable trust. As we've seen, in a blockchain application, the data that will be recorded on-chain includes valid transactions, state changes, receipt values from Tx execution, logs of events emitted, and a few related details.

Do not define a traditional database in a smart contract. If you do, there will be multiple replicated databases, one in each node. Also, do not try to port a database of a centralized system into a smart contract. You don't explicitly say "Record this" and "Record that." The blockchain infrastructure in the background records most of the on-chain data.

So how should you store on-chain data? Because smart contracts facilitate on-chain recording, you'll design your contracts with only the functions and data needed for recording a hint of what happened off-chain.

> **DESIGN PRINCIPLE 8:** Design smart contracts with only the functions and data needed for enforcing rules, compliance, regulation, provenance, logs for real-time notifications, and timestamped footprints and messages about offline operations.

You can think of the smart contract as being a rules engine, an enforcer of rules, and a guardian of related data and messages. If you want to prove that you bought a Ferrari, you don't upload all your photographs of the car to the blockchain (via smart contract); instead, you store the hash of the title or index of your (off-chain) Ferrari photo album. People can use this on-chain hash to locate and access your collection of Ferrari photos off-chain.